

VeriDNS: Incremental Distributed Verification of DNS Configurations

anonymous author

Abstract

The Domain Name System (DNS) is a critical Internet component, yet it remains vulnerable to misconfigurations that can cause widespread outages. Existing centralised validation tools suffer from performance bottlenecks. In contrast, distributed validation tools cannot support incremental validation, making it struggle to handle both the scale and the high frequency of configuration updates common in modern infrastructures. In this work, we present VeriDNS, a distributed framework designed to address these twin challenges. VeriDNS models DNS resolution as a formalized resolution state graph, an abstraction that is inherently suitable for parallel, distributed validation. Crucially, this model enables highly efficient incremental verification: configuration changes are translated into localized graph updates, eliminating the need for costly full re-computations. By embedding formal semantics into the validation process, VeriDNS ensures both syntactic and semantic correctness. Our experimental results demonstrate that VeriDNS not only detects a broader class of misconfigurations than state-of-the-art tools but also achieves significant performance gains, showcasing the power of its distributed and incremental architecture.

1. Introduction

The Domain Name System (DNS), formalized in RFCs 1034 [21] and 1035 [22], serves as a cornerstone of the Internet’s infrastructure. Its function as a scalable, hierarchical naming system that translates human-readable domain names into machine-routable IP addresses underpins virtually all global digital services. However, the distributed architecture that grants DNS its resilience also renders it inherently susceptible to misconfigurations [23, 31, 34]. These errors, ranging from missing records to subtle aliasing and delegation

flaws, can precipitate significant service disruptions and security vulnerabilities [3, 12, 7, 26, 20]. The persistence of such high-stakes failures is not merely theoretical; a misconfiguration of “akam.net” as “akam.ne” recently exposed a MasterCard DNS server for five years, creating a long-lived attack vector [17]. Similarly, a Cloudflare outage, triggered by a misconfiguration that impacted DNS resolution, caused widespread Internet disruptions, underscoring the cascading impact of such failures [9].

The persistence of these critical issues highlights a long-standing challenge in network operations. In the nascent era of the Internet, approaches to DNS verification were largely reactive and simplistic. Administrators relied on utilities like “dig” [4] and “nslookup” [19] for ad-hoc analysis, while static analysis tools such as “dnslint” [18] and BIND’s “named-checkzone” [13] focused on validating the syntactic correctness of individual zone files. While useful for catching formatting errors, these methods were fundamentally incapable of reasoning about the semantic behavior of the resolution process across multiple, interacting zones. Their inability to detect logical errors left a critical gap as the Internet’s complexity began to grow.

The manifest inadequacy of these syntax-focused tools necessitated a paradigm shift towards more principled and exhaustive analysis. This led to the development of formal verification methods, a significant leap from passive detection to proactive, model-based correctness proofs. Tools such as GROOT [14] centralized collection of all configuration files, spearheaded this movement by systematically modeling the DNS resolution process. By simulating all potential query paths, GROOT could proactively identify deep, logical errors like inconsistent delegations and resolution loops before deployment, enabling a far more rigorous validation.

Yet, as the Internet’s scale and dynamism continued to expand, the very exhaustiveness that made these formal methods powerful also revealed their critical limitations. Built on a centralized data collection, tools like GROOT encountered significant computational and scalability bottlenecks when analyzing the vast, interdependent DNS zones. While subsequent research explored alternative directions—such as Maude, which focused on DDoS resilience [16], or Octopus [27], which proposed a distributed architecture—a key challenge remained unaddressed. Octopus, despite tackling distributed verification, did not offer an efficient solution for incremental verification. In modern, dynamic environments where configurations are updated frequently, the necessity of re-verifying the entire state from scratch for each minor change renders such systems impractical.

This evolutionary path has brought the field to a critical juncture, leaving fundamental challenges unsolved: scalability in distributed environments and efficiency for handling frequent, incremental updates. The central question thus becomes: How can we verify DNS configurations in a manner that is both massively scalable and inherently incremental?

In this paper, we propose VeriDNS, a distributed verification framework designed to answer this question. VeriDNS introduces a novel graph-based abstraction of DNS resolution behavior, which formally represents the resolution process as a resolution state graph. This model is not only inherently suited for distributed computation but also naturally accommodates incremental validation. By doing so, VeriDNS enables scalable, high-performance verification.

Key Advancements Our approach introduces several key improvements over the state of the art:

- **Scalability:** The VeriDNS model is optimized for distributed environments, ensuring that large-scale DNS configurations can be verified without the performance bottlenecks of centralized systems.
- **Incremental Verification:** Our graph-based framework naturally supports incremental validation. Configuration changes translate to localized updates on the graph, enabling re-verification at a microsecond level.
- **Results:** Our experimental evaluation demonstrates that VeriDNS finds a broader range of distributed errors than the centralized GROOT and achieves superior performance than the distributed Octopus, particularly showcasing its powerful incremental verification capabilities.

Through this work, we provide a powerful and practical tool for ensuring the correctness of DNS configurations in today’s complex and dynamic Internet infrastructure.

2. Background and Motivation

Preliminaries. The DNS is a scalable, decentralized database that resolves domain names into IP addresses and other resource identifiers. Almost all access to internet resources begins with a DNS request. For example, when a user visits a website such as `www.google.com`, the browser first issues a

nameserver:a.gtld-servers.net.						
\$ORIGIN com.						
campus.com.	IN	NS	ns1.campus.com.	(a)		
campus.com.	IN	NS	ns2.campus.com.	(b)		
ns1.campus.com.	IN	A	17.17.17.31	(c)		
ns2.campus.com.	IN	A	17.17.17.32	(d)		

nameserver:ns1.campus.com.						
\$ORIGIN campus.com.						
campus.com.	IN	NS	ns1.campus.com.	(e)		
campus.com.	IN	NS	ns2.campus.com.	(f)		
ns1.campus.com.	IN	A	17.17.17.31	(g)		
ns2.campus.com.	IN	A	17.17.17.32	(h)		
www.campus.com.	IN	A	63.16.72.7	(i)		
*.campus.com.	IN	A	204.16.2.7	(j)		

nameserver:ns2.campus.com.						
\$ORIGIN campus.com.						
campus.com.	IN	NS	ns1.campus.com.	(k)		
campus.com.	IN	NS	ns2.campus.com.	(l)		
ns1.campus.com.	IN	A	17.17.17.31	(m)		
ns2.campus.com.	IN	A	17.17.17.32	(n)		
www.campus.com.	IN	A	63.16.72.7	(o)		
*.campus.com.	IN	CNAME	www.campus.com.	(p)		

Figure 1: Example zone files for three nameservers: `a.gtld-servers.net`, `ns1.campus.com`, and `ns2.campus.com`. The query $\langle \text{score.campus.com}, A \rangle$ has two possible resolution paths: one traversing records $[(a), (c), (j)]$, and another following $[(b), (d), (p), (o)]$.

DNS request to obtain the address of the hosting server. This functionality underpins much of internet communication and ensures that users can access resources via human-readable domain names rather than complex numerical IP addresses. **DNS Namespace.** It is structured in a tree-like fashion, where each node in the tree represents a domain name, and each node is identified by a fully qualified domain name (FQDN) that uniquely specifies its location in the DNS hierarchy. The dot represents the root of the tree (`.`), and from there, the namespace branches into subdomains. For simplicity and ease of reading, we typically use a shorthand notation for domain names (*e.g.*, “`www.google.com`” instead of the fully qualified format, except in special cases where full notation is required).

Resource Record. Zones represent distinct portions of the DNS hierarchy, and each zone contains a set of resource records (RRs) that map domain names to various types of data, such as IP addresses or mail exchange servers. Each RR is commonly modeled as a quintuple $\langle d, t, c, \tau, v \rangle$, denoting the domain name d , type of query t (*e.g.*, `A`, `NS`), class c (*e.g.*, `IN`, `CH`), time-to-live τ , and the value or target v .

DNS Resolution Process. When a client initiates a DNS query to resolve a domain name, it contacts a recursive resolver, often provided by an Internet

Service Provider or services like Google Public DNS. The resolver first checks its cache for valid records from previous queries, returning an immediate response if available. If no cached entry exists, the resolver iteratively queries the DNS hierarchy, starting with a root name server to obtain a referral to the top-level domain (TLD) name server. It follows subsequent referrals, querying each level of authoritative name servers until reaching the server responsible for the target domain's zone. This server returns the requested resource records, such as an IP address for an A record.

Example. Figure 1 provides a simplified yet illustrative example of DNS resolution. A campus operates two authoritative nameservers, `ns1.campus.com` and `ns2.campus.com`. While migrating to a new hosting provider, the zone file on `ns1` has been updated with the new IP address for `score.campus.com`, but `ns2` still retains the outdated configuration. When a client resolver queries `(score.campus.com, A)`, it might receive a response from either `ns1` or `ns2`, depending on factors such as caching, round-robin ordering, or network latency. This leads to inconsistent resolution results for the same query. Specifically, the query `(score.campus.com, A)` can follow two distinct resolution paths: one traversing records [a, c, j] and the other traversing records [b, d, p, o].

Despite its seeming simplicity, the DNS resolution process is fundamental globally, and its configuration remains notoriously prone to errors [1, 24]. Misconfigurations are alarmingly frequent and can arise from manual errors, outdated migration scripts, uncoordinated updates across authoritative servers, and record misalignments in delegations. Such issues can result in name resolution inconsistencies, partial reachability, or complete service outages. Notable outages at major providers like GitHub, Spotify, or Cloudflare have been attributed to subtle but critical DNS misconfigurations [3, 29, 26, 9, 17, 7].

Existing Tools. Monitoring-based tools often fail to uncover such bugs due to the distributed and non-deterministic nature of resolution paths [32, 6, 25]. These tools, such as “dnschecker” [5] and “whatsmydns” [28], are designed to check the global propagation of DNS records and monitor their availability and performance from various locations. While valuable for observing the live state of the DNS infrastructure, their reliance on periodic checks of specific record types (*e.g.*, A, MX) means they are reactive. They can identify when a service is unreachable, but cannot proactively detect underlying logical flaws in zone file configurations that might only manifest under specific, and often rare, query patterns. Their core limitation is that they do not model the DNS

resolution process itself, and therefore cannot reason about the correctness of delegation chains or forwarding loops before they cause an outage.

Static and formal analysis tools offer a more proactive approach by directly inspecting zone files to find misconfigurations before deployment. However, early static analysis tools were often limited to simple syntactic checks, more advanced formal methods have led to powerful verification tools like GROOT [14], which exhaustively analyzes DNS configurations by partitioning the vast space of possible queries into a finite set of equivalence classes. Yet, as the Internet’s scale continued to expand, the very exhaustiveness that made these formal methods powerful also revealed their critical limitations. Built on a centralized data collection model, tools like GROOT encounter significant computational and scalability bottlenecks when analyzing vast, interdependent DNS zones. The author of GROOT also shows in [15] that, when the volume of zone files becomes excessive, verification time explodes exponentially.

Subsequent research explored alternative directions to address these shortcomings. Octopus [27] proposed a distributed architecture to tackle the challenge of verification across multiple administrative domains. Despite this advance, a key challenge remained unaddressed: Octopus did not offer an efficient solution for incremental verification. In modern, dynamic environments where configurations are updated frequently, the necessity of re-verifying the entire state from scratch for each minor change renders such systems impractical and underscores the need for a new approach.

Formalizing Resolution as Graph. The core limitation of existing tools, as discussed, is their failure to model the actual resolution process—a dynamic sequence of state transitions across distributed zones. To overcome this, we argue for a fundamental shift in perspective: instead of viewing zone files as static tables, we must model the query resolution path itself as a traversal through a state-transition graph. This abstraction allows us to capture the operational semantics of DNS behavior, which is essential for uncovering complex, interactive bugs.

We formalize this concept as a resolution graph. In this graph, each node represents a logical state in the resolution of a query $\langle d, t \rangle$, d is the query name, t is the query type, and each directed edge represents a state transition driven by a specific RR. The states can be categorized as follows:

- **Initial state** (s_i): Where a resolver begins processing a query for a given zone.

- **Intermediate state** (s_m): Represents a non-terminal step where the query is rewritten or delegated via records like CNAME or NS.
- **Final state** (s_f): Where the resolution terminates, yielding a definitive answer such as an IP address, or an authoritative NOERROR (with no data) or NXDOMAIN response.

This graph-based formalization provides a powerful analytical framework. It enables us to rigorously reason about critical properties of DNS configurations that are otherwise elusive to monitoring or purely static approaches:

- *Determinism*: Do all valid resolution paths for the same query converge to the same final state? Divergence can indicate serious misconfigurations.
- *Termination*: Can any query enter a loop, thus never reaching a final state? This identifies bugs like cyclic dependencies in CNAME or DNAME records.
- *Reachability*: Are there valid records in a zone file that can never be reached by any legitimate query, indicating dead or orphaned configurations?

Analyzing these properties is critical for ensuring the reliability and security of DNS, especially in distributed environments where zone files are managed independently.

Crucially, this method is inherently suited to address the key challenges of distribution and incremental updates that hobbled previous systems. By decomposing the global resolution process into a collection of interconnected per-zone subgraphs, it provides a natural foundation for distributed verification. Furthermore, a change to a single RR corresponds to a localized modification of the graph, enabling efficient incremental analysis without re-verifying the entire state from scratch. This abstraction forms the cornerstone of our proposed framework, VeriDNS, which we detail in the next section.

3. Design

At its core, VeriDNS is a distributed verification framework that operates by transforming DNS configurations into a novel structure we call the resolution state graph (RSG). This graph-centric abstraction makes the implicit,

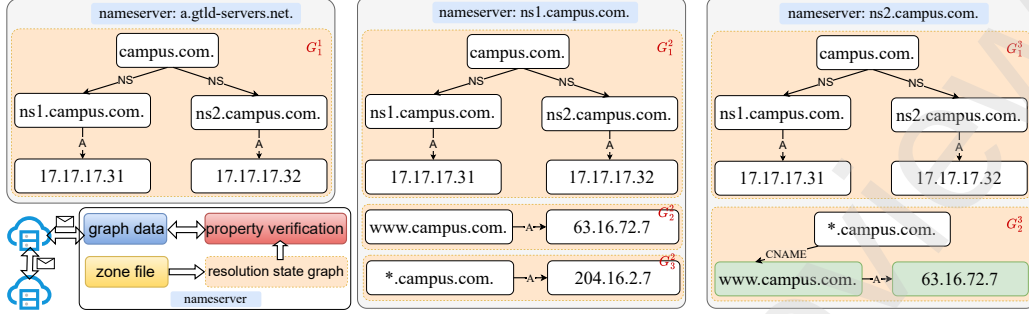


Figure 2: A simple example to illustrate how our framework performs RSG construction, as well as self-checking and cross-checking validations.

operational semantics of DNS resolution explicit (§3.1). This model enables a powerful two-phase validation process (§3.2): *self-checking* for intra-zone correctness and *cross-checking* for inter-zone consistency. To ensure practicality in dynamic environments, the framework is designed to handle incremental updates efficiently, maintaining scalability even with frequent configuration changes (§3.3).

3.1. Resolution State Graph

Zone files, being central to DNS configurations, are often represented in a semi-structured format designed primarily for performance, but not for formal reasoning or analysis. To bridge this gap and enable rigorous analysis, we aim to model each RR in a zone file as a state transition in a formal state dependency graph. This approach facilitates the use of graph-theoretic and logical operations to analyze the interactions between the records. Inspired by previous work [30, 33, 14], we define the RSG as a directed multi-graph, where each edge represents a resolution or transition from one state to another, modeling DNS resolution as a sequence of state transitions. This representation allows us to reason about semantic properties, coordinate cross-zone analysis, and efficiently analyze resolution behavior.

Formal Definition. We model a DNS zone as a set of RRs, $\mathcal{Z} = \{R_1, R_2, \dots, R_n\}$. The resolution behavior within this zone is represented as a Resolution State Graph (RSG), defined as a labeled, directed multi-graph $G = (V, T, E)$, where:

- V : A set of vertices (or states), where each vertex $v \in V$ represents a unique DNS entity, such as a domain name or a terminal value (e.g.,

an IP address).

- T : A set of transition types, corresponding to the different types of DNS records (*e.g.*, A, NS, CNAME).
- E : A set of directed edges. Each edge $e = (s_i, t, s_j) \in E$ represents a state transition, signifying that the entity $s_i \in V$ can resolve to the entity $s_j \in V$ via a RR of type $t \in T$.

Each RR in the zone is parsed into one or more edges in the RSG. For example, an A record `www.school.com. A 192.0.2.1` corresponds to an edge `(www.school.com, A, 192.0.2.1)`. The resolution process is formalized by a transition function $\delta : V \times T \rightarrow V$, where $\delta(s_i, t) = s_j$ indicates that entity s_i resolves to entity s_j via record type t . For instance, $\delta(\text{www.school.com}, \text{A}) = 192.0.2.1$. This model provides a rigorous mathematical foundation for analyzing DNS configurations while remaining intuitive through its graph-based structure.

Construction Process. Given a zone file $\mathcal{Z}$, the corresponding RSG is built through a structured process that distinguishes between internal and external dependencies.

Intra-Zone Graph Expansion. The process starts by building the graph from all RRs defined within the zone file $\mathcal{Z}$. We initialize an empty graph $G = (V, T, E)$ and iterate through each RR. For each record, parsed as a triple (s_i, t, s_j) representing (owner name, type, rdata): The source entity s_i and target entity s_j are added as vertices to V if they are not already present. A directed edge (s_i, t, s_j) is added to E . This initial loop constructs a graph that captures all explicit, locally-defined resolution paths within the zone.

Identifying External Dependencies. RRs such as NS records (delegations) or CNAMEs pointing to external domains inherently create dependencies on other zones. Our construction process identifies these but does not attempt to resolve them at this stage. When a target entity s_j (*e.g.*, `ns1.another-provider.com`) is not defined within the current zone $\mathcal{Z}$, the vertex s_j is marked as an external stub node. These stubs are critical connection points that serve as the interface for the cross-checking phase described later. This clear separation prevents the construction process from overstepping its bounds into inter-zone verification.

Graph Convergence The RSG is constructed iteratively by processing the RRs in a DNS zone $\mathcal{Z} = \{R_1, R_2, \dots, R_n\}$. A graph expansion operator F adds vertices and edges to the RSG $G = (V, T, E)$ based on the RRs. The

process converges when a full pass over the RRs produces no new vertices or edges, i.e., when $F(G) = G$, indicating that all possible state transitions have been captured. The construction process is guaranteed to terminate. Since the number of RRs in a zone is finite, and each record is processed a constant number of times, the algorithm generates a finite set of vertices and edges, adding only entities explicitly defined in the RRs. Semantic errors, such as resolution loops, are detected during analysis of the resulting graph, not as a precondition for its construction. This convergent RSG provides a rigorous foundation for validating DNS configurations and supports advanced features like incremental updates and cross-zone validation in subsequent analyses.

3.2. Property Verification

Once the RSG is constructed, we perform property verification to detect semantic and structural issues in DNS configurations. This verification process is divided into two complementary modes: self-checking and cross-checking. Each mode is underpinned by rigorous formal semantics, which ensure that the checks are both precise and automatable.

Self-checking. The self-checking phase is designed to validate the internal correctness of a single DNS zone by performing local, semantics-aware analysis over its RSG. Given an RSG $G = (V, T, E)$ constructed as described in §3.1, we traverse this graph and use a suite of logical predicates to identify semantic violations. These predicates are defined based on the graph structure and capture common classes of DNS misconfigurations, such as CNAME loops, missing glue records, and improper delegations. The following key properties are checked via graph-based predicates:

- **Missing Glue Record:** For each NS delegation to a nameserver, if the target nameserver's domain name lies under the current origin and does not have an accompanying A or AAAA record, a glue record is missing: $(s_i, \text{NS}, s_j) \in E \wedge s_j \in \text{origin}(z) \wedge \nexists (s_j, t, s_k), t \in \{\text{A}, \text{AAAA}\}$.
- **Lame Delegation:** If an NS record points to a nameserver that does not respond authoritatively for the delegated zone, the delegation is considered lame: $(s_i, \text{NS}, s_j) \in E \wedge \text{query}(s_j) = \text{REFUSED}$.
- **Rewrite Loop:** A CNAME chain must eventually terminate in a non-CNAME record. Let $\pi = \{(s_1, \text{CNAME}, s_2), \dots, (s_k, \text{CNAME}, s_{k+1})\}$ be a path in G . If $\exists i$ such that $s_i = s_{k+1}$, a CNAME loop

is detected: $\exists \pi \subseteq G$ s.t. $s_1 = s_{k+1} \wedge \forall (s_j, t, s_{j+1}) \in \pi, t = \text{CNAME}$. Suppose a domain name is rewritten to another domain name that is not under the current origin. In that case, the final rewritten domain name will carry this path and initiate the corresponding DNS resolution, and which checking will be handed over to cross-checking.

- **Rewrite Blackholing:** For rewritten queries via CNAME or DNAME, if the target domain does not exist, the system may leak NXDOMAIN errors: $\exists \pi = \{(s_i, t, s_j), \dots\}, t \in \{\text{CNAME}, \text{DNAME}\}, \text{query}(s_j) = \text{NXDOMAIN}$. Suppose a domain name is rewritten to another domain name of its current origin, or the current nameserver can handle the rewritten domain name, then only linear time consumption is needed to obtain the corresponding value on the current domain nameserver; otherwise, it will be further handed over to cross-checking. For the DNAME type, after ensuring that the conversion of $(s_i, t, s_j), (t \in \text{DNAME})$ does not loop, no more query requests will be sent. The verification result of the rewritten domain name can be directly used to reduce repetitive verification

These predicates are directly evaluated on the graph G using depth-first traversal (DFS), which ensures full coverage of resolution chains while detecting both cycles and terminal violations. And it is not necessary to exhaust all query requests.

Cross-checking. While self-checking ensures the internal consistency of a single DNS zone, DNS is inherently distributed, and correctness depends not only on individual zone files but also on the consistency of delegation chains and glue records across zones. Many misconfigurations arise from interactions across multiple zones maintained by different administrative entities. The *cross-checking* phase targets such inter-zone issues by validating delegation relationships and shared dependencies between zones. This process leverages the previously constructed RSGs of multiple zones, applying inter-zone predicates to identify delegation inconsistency, cyclic zone dependency, rewrite loops, overflow, and rewrite blackholings.

Delegation Validation. Let G^1 and G^2 be the RSGs of two distinct zones. A delegation edge $(s_i^1, \text{NS}, s_j^1) \in G^1$ implies that the domain s_i^1 is delegating to a nameserver s_j^1 . Cross-checking verifies: The corresponding address record of s_j^1 must exist in either G^1 (as glue) or G^2 (as authoritative data). The authoritative answer from G^2 for s_j^1 must match the delegation record in G^1 : $\exists (s_j^2, t, s_k^2) \in G^2, t \in \{\text{A}, \text{AAAA}\}, s_k^2 = s_j^1$.

Shared Path Reuse. To improve efficiency, our cross-checking process reuses resolution paths computed during self-checking. We model the DNS resolution process as a functional mapping, where multiple domain queries may resolve to a single canonical endpoint. Suppose two queries, q_1 (from zone Z_1) and q_2 (from zone Z_2), both resolve to the same canonical name, c , for instance, through a chain of CNAME records. This relationship can be expressed as: $\text{resolve}(q_1) = c$ and $\text{resolve}(q_2) = c$. Given this many-to-one mapping, the validation of both q_1 and q_2 becomes dependent on the validation of the common target, c . Instead of independently performing the full resolution and validation for each query, we introduce an optimization. The resolution path and validation status of the canonical target c are computed once. Consequently, any query that resolves to c can reuse this cached validation result. This logic can be formally stated as follows: If $\text{resolve}(q) = c$, then $\text{valid}(c) \implies \text{valid}(q)$. This optimization significantly reduces query overhead and computational redundancy in large-scale deployments, particularly where many domains redirect to a common canonical target.

3.3. Incremental Verification

In modern operating environments, DNS configurations are not static but are frequently updated to add services, change IP addresses, or modify delegation policies. After each minor change, revalidating the entire configuration from scratch is not only computationally expensive but also fails to provide the timely feedback required. Therefore, an efficient incremental validation mechanism is not only a performance optimization but also a critical feature indispensable in actual deployments. The key insight behind our incremental approach is that most configuration changes have a localized impact. The explicit dependency structure of the RSG allows us to precisely identify and isolate this impact. Instead of treating the zone file as an opaque block of text, we can map a change to a specific set of vertices and edges in the graph and then analyze how this local change propagates through the system.

Our incremental verification process for DNS zone modifications unfolds in three distinct stages. First, any change to a zone file—whether an insertion, deletion, or update of an RR—is translated into a set of elementary graph operations, forming a delta set ΔG . This set encapsulates the direct modifications to the RSG. Second, to capture the full impact of these changes, including cascading effects, the algorithm performs a comprehensive impact analysis. It conducts a two-way graph traversal starting from the nodes in

ΔG : a backward traversal identifies all altered predecessor paths, while a forward traversal uncovers all dependent successor paths. The union of these paths defines the affected subgraph. Finally, VeriDNS applies its full suite of property checks, as detailed in §3.2, solely to this focused subgraph. By targeting only the affected portions of the graph, this approach efficiently verifies both direct and indirect consequences of the update with minimal computational overhead, addressing the challenge of verifying dynamic DNS configurations.

4. An Example

To concretely illustrate the workflow of VeriDNS, we will walk through a simplified scenario based on the DNS configuration for a hypothetical ‘campus.com’ zone, as depicted in Figure 2.

Building the RSG. By reading the zone file line by line and converting each line into a triple (s_i, t, s_j) , we can construct the RSG. For example, the record ① is converted to $(campus.com, NS, ns1.campus.com)$. If the state node is not present in the RSG, it is added to the graph. Then, an edge labeled *NS* is added between *campus.com* and *ns1.campus.com*. Similarly, for the record ② converted to $(campus.com, NS, ns2.campus.com)$, *ns2.campus.com* is added to the RSG and an *NS* edge is added between *campus.com* and *ns2.campus.com*. By repeating these steps, a complete RSG can be constructed.

Property verification. In the self-checking phase, the verifier initiates a depth-first traversal from nodes with zero in-degree in the RSG. During traversal, the verifier examines each path, along with its edge types and state transitions, to check property-specific conditions. For example, in Figure 2, we obtain the path $(campus.com, NS, ns2.campus.com)$, which satisfies the condition $glue(s_i, s_m)$. If the next node of *ns2.campus.com* is empty, without an *A* record or *AAAA* record, it indicates a missing glue record.

The cross-checking involves transforming the specific graph data according to the property that needs to be checked. The necessary data is then sent to the appropriate addresses for verification. For example, to check for delegation inconsistency, glue-related data G_1^1 for “ns1.campus.com” is extracted and sent to IP address “17.17.17.31”. The address “17.17.17.31” receives the data and the property to be checked, and then performs property verification using the local RSG data G_1^2 . If the remote server is unreachable,

misconfigured, or returns SERVFAIL, a network-level or protocol-level error is reported.

Incremental Verification. Now, assume an administrator modifies record ④, changing the IP address for “ns2.campus.com” from “17.17.17.32” to “17.17.17.33”. Instead of re-validating everything, VeriDNS’s incremental engine takes over. The change is mapped to the deletion of the old A record edge and the addition of a new one. From this focal point, the impact analysis begins. A backward traversal from the “ns2.campus.com” vertex immediately identifies the incoming NS record ⑥ from “campus.com” as an affected predecessor path. A forward traversal from the new IP node finds no further outgoing dependencies. The resulting affected subgraph is thus precisely identified. The verifier then re-runs its checks only on this small subgraph: the self-check confirms the glue record is still valid with the new IP, and the cross-check is re-triggered, sending the updated delegation proof to the relevant nameservers to ensure global consistency is maintained.

5. Evaluation

We evaluate the effectiveness and efficiency of VeriDNS through five key questions: **Q1.** Can VeriDNS detect more DNS misconfigurations than existing methods? **Q2.** Is VeriDNS more efficient than centralized tools such as GROOT in verifying configuration errors? **Q3.** Does the distributed and reusable nature of VeriDNS significantly reduce verification overhead at scale? **Q4.** How effective is incremental verification of the VeriDNS? To answer these questions, we conduct four sets of experiments: *single-device experiments* (§5.1) to evaluate the bug detection ability of self-checking, *multi-device experiments* (§5.2) to measure the time efficiency of distributed verification, *compare experiments* (§5.3) to measure the verification reusability, and *incremental experiments* (§5.4) to measure the efficiency of incremental verification.

5.1. Single-device Experiments

Environment Setting. All experiments were conducted on a machine with a Tesla T4 GPU, Intel(R) Xeon(R) Silver 4210 CPU, and 64GB of memory. VeriDNS is implemented in Python (~1500 lines of code) using Ubuntu 18.04 LTS, Python 3.9.0, dnspython 2.6.1, and networkx 3.2.1.

Dataset. We use the Census dataset [2], consistent with the evaluation in GROOT. This dataset captures real DNS query data from 2012–2013,

Table 1: The network problems were found when checking the bugs.

Problem description	Bug numbers
Network is unreachable	3
The DNS operation timed out.	461
The server answered SERVFAIL	3,665
Connection timed out	2,534
Connection refused	356
No route to host	183
Connection reset by peer	887

filtered and processed to maintain DNS hierarchy. We select 10,000 domains, corresponding to over 22,000 zone files. .

Metric. We compare the number of misconfiguration bugs detected in individual zone files using VeriDNS, GROOT, and Octopus. This assesses the coverage and precision of our self-checking mechanism.

Results. Figure 3 shows that VeriDNS detects significantly more bugs than GROOT. For example, VeriDNS identifies over 11,000 lame delegations and 13,000 NXDOMAIN errors, while GROOT fails to detect any due to its limited static analysis. This result answers **Q1** affirmatively: VeriDNS captures a broader range of misconfigurations by leveraging graph traversal and dynamic query analysis. In addition, Table 1 presents runtime DNS errors such as SERVFAIL and timeouts, which provide further evidence of real-world misbehavior that VeriDNS exposes. Since Octopus directly faces distributed processing and there is no single file check, no errors were detected.

Answer to Q1. VeriDNS outperforms GROOT in bug coverage by combining dynamic resolution with state-based modeling. This enables it to uncover not only structural but also behavioral misconfigurations, achieving broader validation.

5.2. Multi-device Experiments

Dataset. We use a campus-collected dataset with 49 zone files, each containing around 10,000 resource records (490,000 total records). These zones exhibit complex features and realistic inter-zone delegations and dependencies.

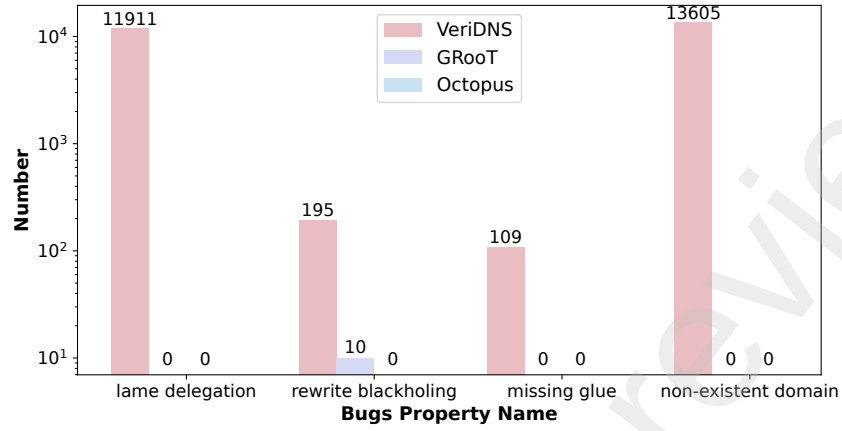


Figure 3: The results of checking bugs on the single file.

Environment Setup. Our experimental setup involved four machines: each server hosted part of the zone files, and when one machine acted as a client and send a verification query, the other three machines acted as distributed verification servers.

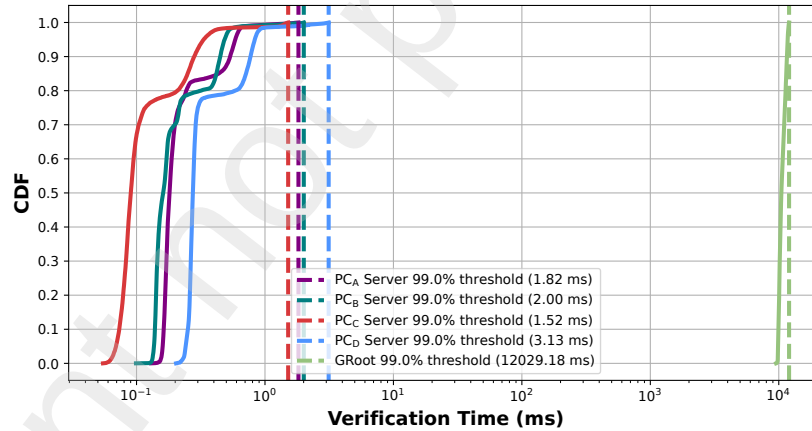


Figure 4: The verification time generated by multi-server and GROOT.

Metric. The client sent 10,000 verification queries to evaluate two common cross-zone properties: delegation inconsistency and missing glue records. We measured verification latency and compared it against the centralized GROOT baseline. To provide a comprehensive performance analysis, we assessed server-side processing time, defined as the duration a server takes to

process a single verification request upon receipt, and end-to-end client latency, which measures the total time from when the client sends a query to when it receives the final verification result. For GROOT, operating on a centralized model, these two metrics are identical, as all processing occurs on a single machine after data collection.

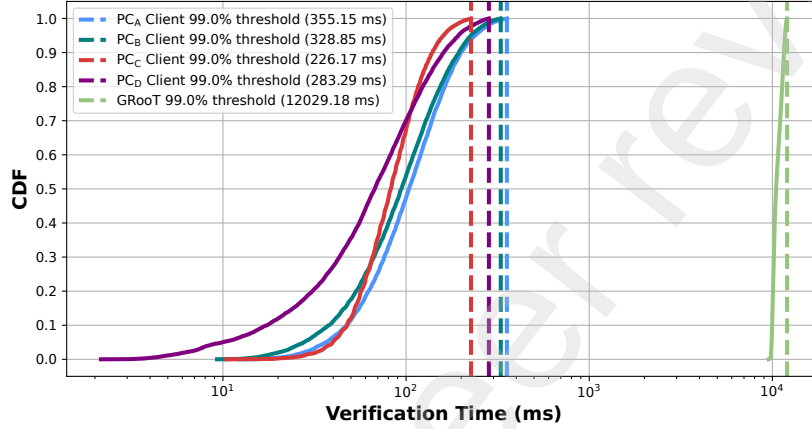


Figure 5: The verification time generated by multi-client and GROOT.

Result. The expensive results show that compared with GROOT, our framework takes less time to verify the DNS misconfiguration, show a dramatic performance improvement offered by VeriDNS’s distributed architecture. In Figure 4, 99% the max verification time used on the server for VeriDNS $\leq 3.132\text{ ms}$ compares to GROOT $\leq 12,029.185\text{ ms}$. In stark contrast, a performance difference of nearly four orders of magnitude for the same task. Even when considering end-to-end communication delays, in Figure 5, 99% the max verification time used on the client for VeriDNS $\leq 355.153\text{ ms}$ compares to GROOT $\leq 12,029.185\text{ ms}$.

Answer to Q2. VeriDNS is substantially more efficient than centralized tools. The performance gain is a direct result of our design: by distributing the verification load and leveraging the RSG to avoid redundant computations, VeriDNS eliminates the single-point bottleneck inherent in centralized systems like GROOT, where all zone files must be collected.

5.3. Compare Experiments

Metric. For VeriDNS, we break down the time into: I/O time, construction time, and verification time. For GROOT, we measure the total end-to-end

time as it integrates all stages.

Results. As shown in Figure 6, VeriDNS exhibited slightly higher latency for I/O and build operations due to programming language performance constraints. However, its total per-device processing time was significantly lower than that of the centralized GROOT baseline. Specifically, GROOT completed verification in 16,473.5 ms, whereas the most time-intensive DNS server in VeriDNS required only 11,458.8 ms. Additionally, as shown in Figure 7, compared to the state-of-the-art distributed architecture Octopus, VeriDNS completed verification in 407 ms, while Octopus required 512 ms. This is due to our memoization of verification results, which eliminates the need for multiple verifications.

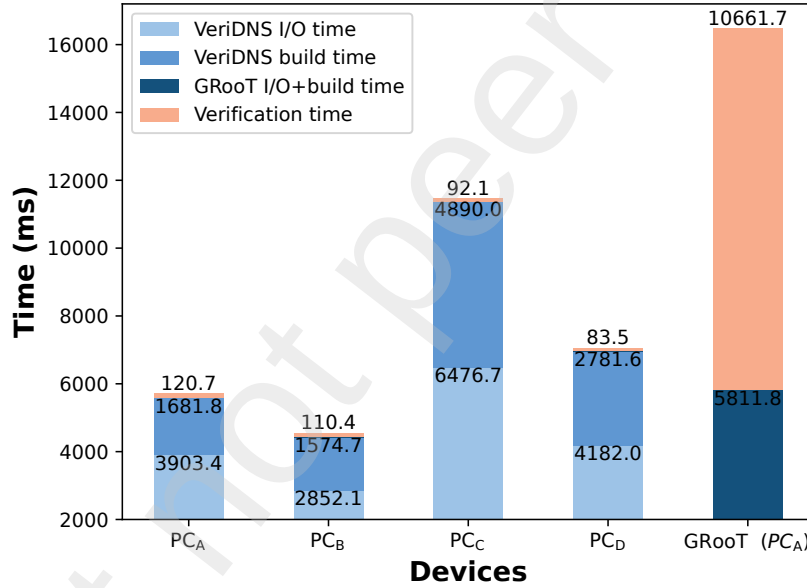


Figure 6: The breakdown of the mean usage time on each device.

Answer to Q3. The distributed design of VeriDNS scales efficiently across multiple devices by reusing resolution subgraphs during cross-checking. For example, when multiple queries share a common resolution suffix (*e.g.*, CNAME chains leading to the same target), VeriDNS reuses the downstream subgraph rather than re-evaluating it. This reuse minimizes duplicated computation, amortizes query costs, and significantly reduces end-to-end overhead when verifying large-scale zone files across distributed environments.

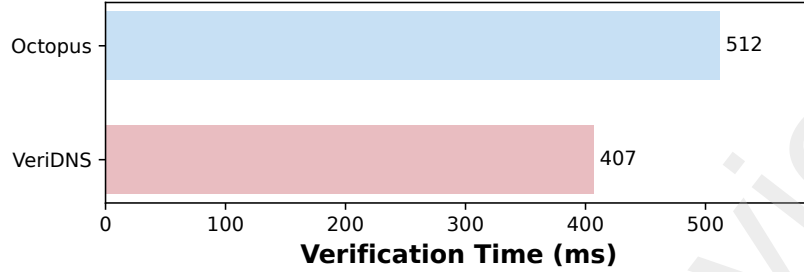


Figure 7: The verification time for different models.

Summary. VeriDNS detects more DNS misconfigurations than GROOT and does so more efficiently, both in single-device and distributed environments. Its graph-based structure, reuse of state transitions, and modular verification pipeline provide both functional and performance advantages.

5.4. Incremental Experiments

Dataset. We use the same dataset as in §5.2. From this dataset, we select a representative zone file containing over 10,000 RRs. To evaluate incremental verification, we simulate updates by injecting 10,000 new RRs under three scenarios: (1) **A** records, (2) **CNAME** records, and (3) randomly selected record types. The objective is to assess whether VeriDNS can efficiently validate changes without re-verifying the entire zone and to check whether errors such as blackholing are introduced.

Results. Figure 8 summarizes the verification latency under the three update scenarios. VeriDNS achieves microsecond-level performance across all cases: 99% the verification time is $\leq 2.39 \times 10^4$ ns for **A** RR and $\leq 3.16 \times 10^4$ ns for **CNAME** RR insertions, and $\leq 3.34 \times 10^4$ ns for random updates. These results demonstrate that incremental validation remains lightweight even under large-scale changes, confirming the efficiency of our change-propagation mechanism.

Answer to Q4. Incremental verification in VeriDNS is highly effective. By localizing validation to only the affected subgraph, the framework avoids redundant recomputation and maintains near-constant latency regardless of update type. This validates the practicality of VeriDNS for real-world DNS environments where frequent updates are common.

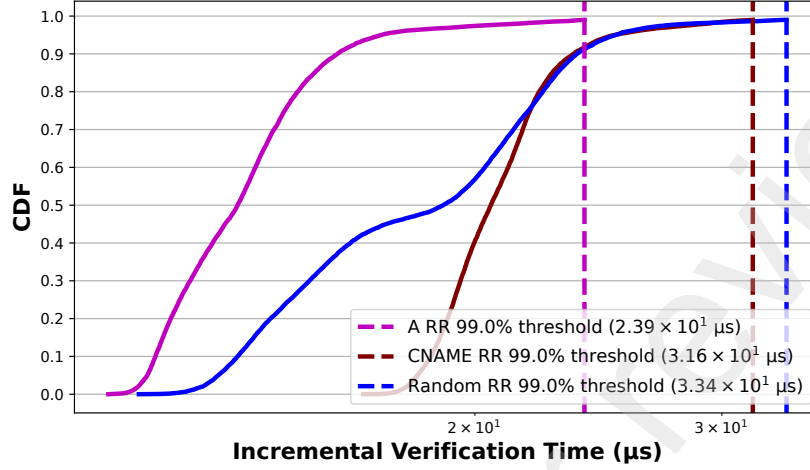


Figure 8: The incremental verification time required for updated RR.

6. Discussion

Scalability and Deployment. VeriDNS adopts a decentralized architecture and leverages state graph abstraction to support scalable DNS configuration validation. Our results (§5.2) demonstrate that verification tasks can be distributed across multiple devices without requiring centralized access to all zone files. This allows VeriDNS to scale gracefully with large DNS deployments, especially in multi-administrator environments such as CDNs, academic networks, or enterprise IT infrastructures.

Formal Semantics Enable Precision. Unlike purely heuristic-based approaches, VeriDNS defines misconfiguration properties through formal semantics. This rigor allows us to verify DNS behaviors with well-defined correctness criteria, avoiding ambiguity or false positives. Furthermore, this formalism enables future extension to support custom policy checks or new record types (*e.g.*, SVCB/HTTPS records).

Limitations and Future Work. While VeriDNS shows strong results, there remain challenges. First, our current model assumes consistency in DNS responses during verification time, which may not hold under rapidly changing environments (*e.g.*, short TTLs or load-balanced DNS). Second, although our framework is compatible with most RFC-compliant zone files, certain edge cases such as DNSSEC validation, DoT [10], DoH [8], and DoQ [11], interactions are currently out of scope.

In future work, we plan to explore: (1) deeper integration with DNSSEC and modern privacy-enhancing DNS protocols, and (2) diagnose misconfigurations, propose and enforce automated repairs in real time.

7. Conclusion

We present VeriDNS, a distributed framework that tackles the key challenges of scalability and incremental efficiency in large-scale DNS verification. At its core, VeriDNS introduces a novel graph-based resolution state abstraction that naturally decomposes the verification task for distributed computation. This model maps configuration changes to localized graph updates, enabling incremental re-verification at microsecond-level latency. Our evaluation demonstrates that VeriDNS not only outperforms the centralized GROOT in finding errors but also its verification performance significantly surpasses existing distributed approaches like Octopus. Ultimately, VeriDNS provides a powerful and practical solution for ensuring DNS correctness and reliability in today’s large-scale, highly dynamic Internet environments.

References

- [1] Yehuda Afek, Anat Bremler-Barr, and Lior Shafir. NXNSAttack: Recursive DNS inefficiencies and vulnerabilities. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 631–648. USENIX Association, 2020.
- [2] DNS Census2013. DNS Census2013, 2013. Accessed: June 16, 2025.
- [3] 1.1.1.1 lookup failures on October 4, 2023. <https://blog.cloudflare.com/1-1-1-1-lookup-failures-on-october-4th-2023>, 2023. Accessed: June 16, 2025.
- [4] dig - DNS lookup utility. <https://linux.die.net/man/1/dig>, 2022.
- [5] DNS Checker. Dns propagation checker - how to check dns propagation globally? <https://dnschecker.org>, 2023.
- [6] F5 Labs Staff. 2023 Identity Threat Report: The Unpatchables. <https://www.f5.com/labs/articles/threat-intelligence/2023-identity-threat-report-the-unpatchables>, 2023. Accessed: August 16, 2025.

- [7] Hazy Hawk gang exploits DNS misconfigs to hijack trusted domains. <https://www.bleepingcomputer.com/news/security/hazy-hawk-gang-exploits-dns-misconfigs-to-hijack-trusted-domains/>, 2025.
- [8] Paul E. Hoffman and Patrick McManus. DNS Queries over HTTPS (DoH). RFC 8484, 2018.
- [9] Julian Horsey. The day the internet died: Inside the global cloudflare dns outage. <https://www.geeky-gadgets.com/cloudflare-dns-outage-2025-impact/>, 2025. Accessed: August 16, 2025.
- [10] Zi Hu, Liang Zhu, John Heidemann, Allison Mankin, Duane Wessels, and Paul E. Hoffman. Specification for DNS over Transport Layer Security (TLS). RFC 7858, 2016.
- [11] Christian Huitema, Sara Dickinson, and Allison Mankin. DNS over Dedicated QUIC Connections. RFC 9250, 2022.
- [12] InfinityFree. Dns outage at ifastnet: Softaculous down. <https://forum.infinityfree.com/t/dns-outage-at-ifastnet-softaculous-down/19374>, 2019. Accessed: June 16, 2025.
- [13] Internet Systems Consortium, Inc. BIND - DNS Software. <https://www.isc.org/bind/>, 2025. Accessed: June 16, 2025.
- [14] Siva Kesava Reddy Kakarla, Ryan Beckett, Behnaz Arzani, Todd Millstein, and George Varghese. Groot: Proactive verification of dns configurations. In *SIGCOMM'20*, pages ACM, 310–328, 2020.
- [15] Siva Kesava Reddy Kakarla, Ryan Beckett, Todd Millstein, and George Varghese. How complex is dns? In *Proceedings of the 20th ACM Workshop on Hot Topics in Networks*, pages 116–122, 2021.
- [16] Si Liu, Huayi Duan, Lukas Heimes, Marco Bearzi, Jodok Vieli, David Basin, and Adrian Perrig. A formal framework for end-to-end dns resolution. In *SIGCOMM'23*, pages ACM, 932–949, 2023.
- [17] MasterCard DNS Error Went Unnoticed for Years. <https://krebsonsecurity.com/2025/01/mastercard-dns-error-went-unnoticed-for-years>, 2025.

- [18] Microsoft. Description of the DNSLint utility. <https://learn.microsoft.com/en-us/previous-versions/troubleshoot/windows-server/description-dnslint-utility>, 2020. Accessed: June 16, 2025.
- [19] nslookup. <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/nslookup>, 2024.
- [20] MikroTik botnet uses misconfigured SPF DNS records to spread malware. <https://www.bleepingcomputer.com/news/security/mikrotik-botnet-uses-misconfigured-spf-dns-records-to-spread-malware/>, 2025.
- [21] P. V. Mockapetris. Domain names - concepts and facilities. RFC 1034, 1987.
- [22] P. V. Mockapetris. Domain names - implementation and specification. RFC 1035, 1987.
- [23] Paul Mockapetris and Kevin J Dunlap. Development of the domain name system. In *Symposium proceedings on Communications architectures and protocols*, pages 123–133, 1988.
- [24] Giovane C. M. Moura, Sebastian Castro, John Heidemann, and Wes Hardaker. Tsunami: exploiting misconfiguration and vulnerability to ddos dns. In *Proceedings of the 21st ACM Internet Measurement Conference*, IMC’21, page 398–418, New York, NY, USA, 2021. Association for Computing Machinery.
- [25] Anju Ramdas and Ramakrishnan Muthukrishnan. A survey on dns security issues and mitigation techniques. In *2019 International Conference on Intelligent Computing and Control Systems (ICCS)*, pages 781–784, 2019.
- [26] Liam Tung. Azure global outage: Our dns update mangled domain records, says microsoft. <https://www.zdnet.com/article/azure-global-outage-our-dns-update-mangled-domain-records-says-microsoft/>, 2019. Accessed: June 16, 2025.
- [27] Yao Wang, Kexin Yu, Ziyi Wang, Kaiqiang Hu, Haizhou Du, Qiao Xiang, Xing Fang, Geng Li, Ruiting Zhou, Linghe Kong, and Jiwu Shu.

- Rethinking dns configuration verification with a distributed architecture. In *APNet 2024*, pages 23–30. ACM, 2024.
- [28] WhatsMyDNS. Global dns checker - how to check dns propagation. <https://www.whatsmydns.net>, 2023.
- [29] Zack Whittaker. A dns outage just took down a large chunk of the internet. <https://techcrunch.com/2021/07/22/a-dns-outage-just-took-down-a-good-chunk-of-the-internet/>, 2021. Accessed: June 16, 2025.
- [30] Qiao Xiang, Chenyang Huang, Ridi Wen, Yuxin Wang, Xiwen Fan, Zaoxing Liu, Linghe Kong, Dennis Duan, Franck Le, and Wei Sun. Beyond a centralized verifier: Scaling data plane checking via distributed, on-device verification. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 152–166, 2023.
- [31] Donghui Yang, Zhenyu Li, and Gareth Tyson. A deep dive into DNS query failures. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 507–514. USENIX Association, July 2020.
- [32] Bojan Zdrnja, Nevil Brownlee, and Duane Wessels. Passive monitoring of dns anomalies. In *Proceedings of the 4th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, DIMVA’07*, page 129–139, Berlin, Heidelberg, 2007. Springer-Verlag.
- [33] Peng Zhang, Xu Liu, Hongkun Yang, Ning Kang, Zhengchang Gu, and Hao Li. Apkeep: Realtime verification for real networks. In *17th USENIX Symposium on Networked Systems Design and Implementation*, pages 241–255, 2020.
- [34] Yunyi Zhang, Baojun Liu, Haixin Duan, Min Zhang, Xiang Li, Fan Shi, Chengxi Xu, and Eihal Alowaisheq. Rethinking the security threats of stale DNS glue records. In *USENIX Security 2024*, pages 1261–1277. USENIX, 2024.